

تحلیل تاثیر تغییرات بر نیازمندی ها

برای این فاز فرض کردیم نیازمندی جدیدی مطرح شده. سامانه باید علاوه بر برگزاری آزمون آنلاین، از قابلیت نظارت هوشمند بر آزمون هم پشتیبانی کند. منظور از این قابلیت نظارت خودکار بر رفتار دانشجو در طول آزمون با استفاده از دوربین، میکروفون و تحلیل رویدادهای سیستم است تا احتمال تقلب کاهش یابد و اعتبار آزمون افزایش پیدا کند.

در نگاه اول ممکن است این تغییر تنها به بخش آزمون مربوط باشد اما پس از بررسی معماری و نیازمندی‌های پروژه متوجه شدیم که این درخواست تقریباً بر تمامی لایه‌های سیستم تاثیر می‌گذارد و صرفاً اضافه کردن چند کلاس جدید کافی نیست. پس تلاش کردیم تاثیر این تغییر را بر سه بخش نیازمندی‌ها، طراحی نرم‌افزار و پیاده‌سازی موجود به صورت جداگانه بررسی کنیم.

تاثیر تغییر بر نیازمندی‌ها

اولین بخشی که تحت تاثیر این تغییر قرار می‌گیرد نیازمندی‌های کارکردی سیستم است. در نسخه اولیه سامانه، آزمون تنها شامل ایجاد سوال، شروع آزمون، ارسال پاسخ‌ها و تصحیح آن‌ها بود اما با اضافه شدن قابلیت نظارت هوشمند، فرایند آزمون پیچیده‌تر می‌شود و رفتار دانشجو نیز به بخشی از اطلاعات قابل پردازش سیستم تبدیل خواهد شد. به همین دلیل نیازمندی‌های جدیدی باید به سیستم اضافه شوند. از جمله فعال بودن دوربین دانشجو در طول آزمون، بررسی قطع شدن تصویر یا صدا، تشخیص خروج کاربر از صفحه آزمون، ثبت هشدار در صورت مشاهده رفتار مشکوک، ذخیره رویدادهای مرتبط با آزمون و ارسال گزارش برای مدرس پس از پایان آزمون. همچنین لازم است مدرس بتواند گزارش کامل نظارت را مشاهده کرده و در صورت نیاز تصمیم نهایی درباره صحت آزمون را اتخاذ کند.

علاوه بر نیازمندی‌های کارکردی نیازمندی‌های غیرکارکردی هم تغییر می‌کنند. از آنجا که اطلاعات تصویری و صوتی کاربران پردازش می‌شود امنیت و محرمانگی داده‌ها اهمیت بسیار بیشتری پیدا می‌کند. همچنین سرعت پردازش رویدادها باید افزایش یابد تا هشدارها تقریباً به صورت لحظه‌ای ثبت شوند.

در کنار این موارد حفظ حریم خصوصی کاربران، قابلیت اطمینان سیستم و امکان پردازش همزمان تعداد زیادی آزمون نیز به عنوان نیازمندی‌های جدید مطرح می‌شوند. در نتیجه متوجه شدیم این تغییر تنها یک قابلیت جدید نیست بلکه باعث گسترش مجموعه نیازمندی‌های اولیه سامانه می‌شود.

تأثیر تغییر بر طراحی سیستم

معماری طراحی شده رو هم مورد ارزیابی قرار دادیم تا مشخص بشه آیا ساختار فعلی پاسخگوی این قابلیت جدید هست یا خیر. یکی از مزیت‌های معماری انتخاب شده این بود که سیستم از ابتدا به صورت سرویس‌گرا و لایه‌ای طراحی شده بود پس برای اضافه شدن قابلیت جدید نیازی به تغییر ساختار کلی سامانه وجود ندارد. با این حال لازم است یک سرویس مستقل با عنوان Proctoring Service به معماری اضافه شود.

این سرویس وظیفه دریافت رویدادهای مربوط به آزمون، تحلیل آن‌ها، ثبت هشدارها و تولید گزارش نهایی را بر عهده خواهد داشت. همچنین این سرویس باید از طریق API Gateway با سایر سرویس‌ها در ارتباط باشد تا ارتباطات همچنان متمرکز و کنترل شده باقی بمانند.

از طرف دیگر با توجه به اینکه در معماری اولیه از الگوی Event-Driven نیز استفاده کرده بودیم تصمیم گرفتیم بسیاری از رویدادهای مربوط به آزمون مانند خروج از صفحه، قطع شدن دوربین، قطع اتصال اینترنت یا تشخیص رفتار مشکوک به صورت Event منتشر شوند و سرویس Proctoring این رویدادها را دریافت و پردازش کند. این تصمیم باعث می‌شود سرویس آزمون همچنان مسئول برگزاری آزمون باقی بماند و منطق مربوط به نظارت وارد آن نشود. در نتیجه اصل Single Responsibility نیز حفظ خواهد شد.

در لایه Domain هم لازم است موجودیت‌های جدیدی مانند ProctoringSession ، Violation و ProctoringReport اضافه شوند تا اطلاعات مربوط به نظارت به صورت مستقل از اطلاعات آزمون نگهداری شوند. همچنین در لایه Repository نیز مخازن جدیدی برای ذخیره گزارش‌ها و تخلفات ایجاد خواهد شد.

در کل این تغییر اگرچه باعث افزایش تعداد سرویس‌ها و کلاس‌ها میشه اما به دلیل معماری مناسب اولیه نیازی به بازطراحی کامل سیستم وجود ندارد و تنها توسعه بخش مربوط به آزمون انجام خواهد شد.

تأثیر تغییر بر کد موجود

کد پیاده‌سازی شده را بررسی کردیم تا مشخص شود چه قسمت‌هایی نیاز به تغییر دارند.

در لایه Domain ، کلاس Exam باید توسعه پیدا کنه تا وضعیت فعال بودن نظارت هوشمند، قوانین آزمون و ارتباط با جلسه نظارت را نگهداری کنه. همچنین کلاس Submission هم باید قابلیت ثبت وضعیت تخلفات احتمالی و ارتباط با گزارش نظارت رو داشته باشد.

در بخش Service ، علاوه بر ایجاد ProctoringService لازمه ExamService هم تغییر کنه تا هنگام شروع آزمون، جلسه نظارت رو ایجاد کنه و در پایان آزمون آن را خاتمه دهد. همچنین در زمان ارسال پاسخنامه، وضعیت گزارش نظارت نیز بررسی بشه تا در صورت وجود تخلف، نتیجه آزمون برای بررسی مدرس علامت‌گذاری شود.

در لایه Controller هم API های جدیدی برای شروع نظارت، دریافت هشدارها، پایان جلسه نظارت و مشاهده گزارش آزمون باید اضافه شوند.

از طرف دیگر سیستم رویدادهای پروژه هم توسعه پیدا می‌کند. در نسخه اولیه، Event ها بیشتر برای ارسال اعلان استفاده می‌شدند اما در نسخه جدید لازم است رویدادهایی مانند CameraDisconnected ، MicrophoneDisabled ، WindowFocusLost و SuspiciousActivityDetected هم منتشر شوند تا توسط سرویس Proctoring پردازش شوند.

همچنین Repository های جدیدی برای ذخیره اطلاعات جلسات نظارت، گزارش تخلفات و هشدارها موردنیاز خواهند بود. این موضوع باعث می‌شود ساختار ذخیره‌سازی نیز نسبت به نسخه اولیه گسترده‌تر شود.

در نهایت رابط کاربری سامانه هم باید تغییر کنه و در صفحه آزمون لازم است وضعیت دوربین، میکروفون، اتصال اینترنت و هشدارهای لحظه‌ای به کاربر نمایش داده شود و قبل از شروع آزمون نیز مجوز دسترسی به دوربین و میکروفون دریافت شود.

جمع‌بندی تحلیل

پس از انجام این تحلیل به این نتیجه رسیدیم که اضافه شدن قابلیت Online Proctoring یکی از تغییرات با اثرگذاری بالا در سامانه هستش چون فقط یک قابلیت جدید به سیستم اضافه نمی‌کنه بلکه بخش‌های مختلف پروژه شامل نیازمندی‌ها، طراحی معماری، مدل دامنه، سرویس‌ها، رویدادها، Repository ها، کنترلرها و رابط کاربری را تحت تاثیر قرار می‌دهد. با این حال یکی از نتایج مثبت این بررسی این بود که معماری انتخاب شده در فاز طراحی از انعطاف‌پذیری مناسبی برخورداره و امکان افزودن این قابلیت بدون ایجاد تغییرات اساسی در ساختار کلی سیستم وجود داره. به همین دلیل توسعه این نیازمندی بیشتر به صورت افزودن سرویس‌ها و مؤلفه‌های جدید انجام میشه و نیازی به بازنویسی بخش‌های اصلی سامانه نداریم. موضوعی که نشان می‌دهد انتخاب معماری اولیه برای چنین سامانه‌ای تصمیم مناسبی بوده است.